Filtered Attention for Long-Context Retrieval

Ada Lovelace, Alan Turing

Abstract

Long-context language models often fail to retrieve facts buried in the middle of their input. We introduce filtered attention, a lightweight scoring layer that prunes irrelevant key blocks before the softmax. On the NeedleBench and LongQA benchmarks, filtered attention improves retrieval recall by 11.4 points at 128k tokens while reducing attention FLOPs by 38%. Code is available at https://github.com/ada/filtered-attention.

1 Introduction

Long-context language models struggle to use information placed in the middle of their input (Smith and Doe, 2024). Let θ denote the parameters of a pretrained transformer, and let k be the number of key blocks kept per query.

We propose filtered attention, which scores every key block and keeps only the top k blocks before computing attention. The threshold τ controls how aggressive the filter is.

Retrieval failures are not uniform across the context window. Facts near the beginning and the end of the input are recovered reliably, while facts in the middle are frequently ignored, a pattern earlier work calls the lost-in-the-middle effect.

We argue that much of this failure comes from attention mass being spread over many irrelevant blocks. When most of the probability mass goes to filler text, the few tokens that matter receive too little weight to influence the output.

Our approach keeps the base model frozen. We add a small scoring layer that looks at each block of keys and decides whether the block is worth attending to at all, which keeps training cheap and the method easy to retrofit.

2 Method

Filtered attention improves retrieval recall by 11.4 points at 128k tokens on NeedleBench, as shown in Table 1.

The scoring layer is trained with a contrastive objective on synthetic retrieval tasks, then evaluated without further tuning on natural documents such as legal contracts, scientific papers, and long chat transcripts.

We also study how the filter interacts with positional encodings. Rotary embeddings remain unchanged, and we find no degradation on short-context tasks where the filter keeps nearly every block.

Finally, we analyze the compute budget. Because pruned blocks never enter the softmax, the savings grow with context length, which makes the method most useful exactly where long-context models are most expensive.

Retrieval failures are not uniform across the context window. Facts near the beginning and the end of the input are recovered reliably, while facts in the middle are frequently ignored, a pattern earlier work calls the lost-in-the-middle effect.

We argue that much of this failure comes from attention mass being spread over many irrelevant blocks. When most of the probability mass goes to filler text, the few tokens that matter receive too little weight to influence the output.

Our approach keeps the base model frozen. We add a small scoring layer that looks at each block of keys and decides whether the block is worth attending to at all, which keeps training cheap and the method easy to retrofit.

The scoring layer is trained with a contrastive objective on synthetic retrieval tasks, then evaluated without further tuning on natural documents such as legal contracts, scientific papers, and long chat transcripts.

We also study how the filter interacts with positional encodings. Rotary embeddings remain unchanged, and we find no degradation on short-context tasks where the filter keeps nearly every block.

Finally, we analyze the compute budget. Because pruned blocks never enter the softmax, the savings grow with context length, which makes the method most useful exactly where long-context models are most expensive.

Method	NeedleBench	LongQA
Full attention	61.2	48.9
Filtered attention	72.6	55.0

3 Related work

Retrieval failures are not uniform across the context window. Facts near the beginning and the end of the input are recovered reliably, while facts in the middle are frequently ignored, a pattern earlier work calls the lost-in-the-middle effect.

We argue that much of this failure comes from attention mass being spread over many irrelevant blocks. When most of the probability mass goes to filler text, the few tokens that matter receive too little weight to influence the output.

Our approach keeps the base model frozen. We add a small scoring layer that looks at each block of keys and decides whether the block is worth attending to at all, which keeps training cheap and the method easy to retrofit.

The scoring layer is trained with a contrastive objective on synthetic retrieval tasks, then evaluated without further tuning on natural documents such as legal contracts, scientific papers, and long chat transcripts.

We also study how the filter interacts with positional encodings. Rotary embeddings remain unchanged, and we find no degradation on short-context tasks where the filter keeps nearly every block.

Finally, we analyze the compute budget. Because pruned blocks never enter the softmax, the savings grow with context length, which makes the method most useful exactly where long-context models are most expensive.

Retrieval failures are not uniform across the context window. Facts near the beginning and the end of the input are recovered reliably, while facts in the middle are frequently ignored, a pattern earlier work calls the lost-in-the-middle effect.

We argue that much of this failure comes from attention mass being spread over many irrelevant blocks. When most of the probability mass goes to filler text, the few tokens that matter receive too little weight to influence the output.

Our approach keeps the base model frozen. We add a small scoring layer that looks at each block of keys and decides whether the block is worth attending to at all, which keeps training cheap and the method easy to retrofit.

The scoring layer is trained with a contrastive objective on synthetic retrieval tasks, then evaluated without further tuning on natural documents such as legal contracts, scientific papers, and long chat

transcripts.

We also study how the filter interacts with positional encodings. Rotary embeddings remain unchanged, and we find no degradation on short-context tasks where the filter keeps nearly every block.

Finally, we analyze the compute budget. Because pruned blocks never enter the softmax, the savings grow with context length, which makes the method most useful exactly where long-context models are most expensive.